# SYSTEM ANALYSIS DOCUMENTATION (SAD)

**UMHackathon 2026**

**umhackathon@um.edu.my**

---

*© UMHackathon 2026*

*Email: umhackathon@um.edu.my*

## 1. *Introduction:*

"**bukit**", inspired by the quote "Sikit-sikit, lama-lama jadi bukit" is designed to empower Malaysian citizens, especially B40 groups, to streamline the process of interpreting complex, fragmented subsidies offered by the Government of Malaysia. We leverage Z.AI's GLM to suggest users the most optimal subsidy that can be applied for their situation depending on each category.

### *1.1. Purpose:*

The system analysis document highlights the technical scope and design-related decision behind the development of bukit.

So, the key element of system that has been covered are as followed:

1. **Architecture**:
    a. Bukit is a cloud native platform. The services are separated into microservices, featuring a React (Vite) frontend, a Node.js (Express) backend, database storage with MongoDB for authenticated users along with their respective profiles, ChromaDB for storage of vector embeddings from markdown files for RAG, and deployed on Vercel.
2. **Data Flows:**
    a. The document will illustrate how the user's request on subsidy flows from the user's client application, and moves through the Express backend API for structural parsing. The backend then implements a RAG (Retrieval-Augmented Generation) pipeline to retrieve relevant snippets of Budget 2026, and supplementary documents. Both of this, along with the user's profile information (eg: income category - B40/M40/T20) is sent to the context of Z.AI's GLM together with crafted system instructions, which returns a structured JSON response of the most relevant subsidy, along with alternatives.
3. **Model Process:**
    a. The model will show the end-to-end workflow of the core features, together with data ingestions and semantic retrieval via RAG, then referenced by the GLM to generate suggestions for subsidies.
4. **Role of Reference:**
    a. This document is a reference for all the developers and testers to enable the overall high-level architecture of this project.

### *1.2. **Background**:*

*The Malaysian government manages a large welfare system designed to support citizens through various economic cycles. However, there is a large lack of information between these funds and the citizens' ability to access them. While the government has digitized many services, the delivery of localized and state specific aid remains manual and uncoordinated, leading many people to not know about them.*

© UMHackathon 2026

1. *Previous Version:*
   *The existing format for discovering government subsidies and aid is very static. The states currently operate and host websites on their own, and rely on simple websites and downloadable files, with some of them still rely on physical manuals as well. Malaysian Citizens must manually search for them with no central database to find the correct subsidies for them.*

2. *Changes in Major Architectural Components*
   *Bukit has three major architectural components, first with LLM: integrating the ILMU-GLM 5.1 model to replace keyword searching. While we also moved from static HTML pages to a React Single Page Application, which allows the UI to render based on JSON responses from AI. We also implemented React Context API coupled with JWT authentication to store the user data like birth year and state across the session, acting as a memory.*

3. *New capabilities that are only introduced in this*
   *The system ensures responses are filtered for the exact eligibility before reaching the screen because of the smart subsidy matching, where it injects the user's state into the LLM prompt. The new architecture also analyses structured JSON from the AI to render the wall of text into small boxes. The users can also just ask simple questions instead of looking around the governments' complex manuals.*

## *1.3. Target Stakeholder*

| ***Stakeholders*** | ***Roles*** | ***Expectations*** |
|---|---|---|
| *Malaysian Citizens* | *Provide data during onboarding and query the AI prompt bar to discover eligible financial aids.* | *A simple and conversational experience, highly accurate, personalized subsidy matches delivered in plain language.* |
| *Development Team* | *Build and maintain the frontend, manage the JWT authentication state.* | *Clear API contracts, strict JSON schema from the LLM to prevent breaking the UI, and an easily maintainable codebase.* |
| *Geographically Remote Citizens* | *Use the platform as a primary discovery tool to bypass the need for physical visits to government offices just for information.* | *A reliable digital platform that removes the time tax and costs of traveling to verify eligibility.* |

© UMHackathon 2026

| | | |
|---|---|---|
| *QA Team* | *Validate the system across different user, test for UI stability.* | *A defined scope and proof that the prompting works safely.* |

## 2. System Architecture & Design

### 2.1. High Level Architecture

#### 2.1.1. Overview:

| Type | Details |
|---|---|
| *System* | Web Application (Responsive Desktop and Mobile UI) |
| *Architecture* | Client-server Cloud Architecture with Microservices approach |

Bukit is primarily structured as a client-server based cloud architecture where it mainly focuses on the primary client. The system consists of a lightweight React (Vite) frontend client that communicates with an asynchronous Node.js (Express) backend via RESTful APIs. The Express backend acts as the central orchestrator, managing secure communication between the client, MongoDB (handles relational user data), and the external Z.AI GLM. All services are deployed on Vercel.

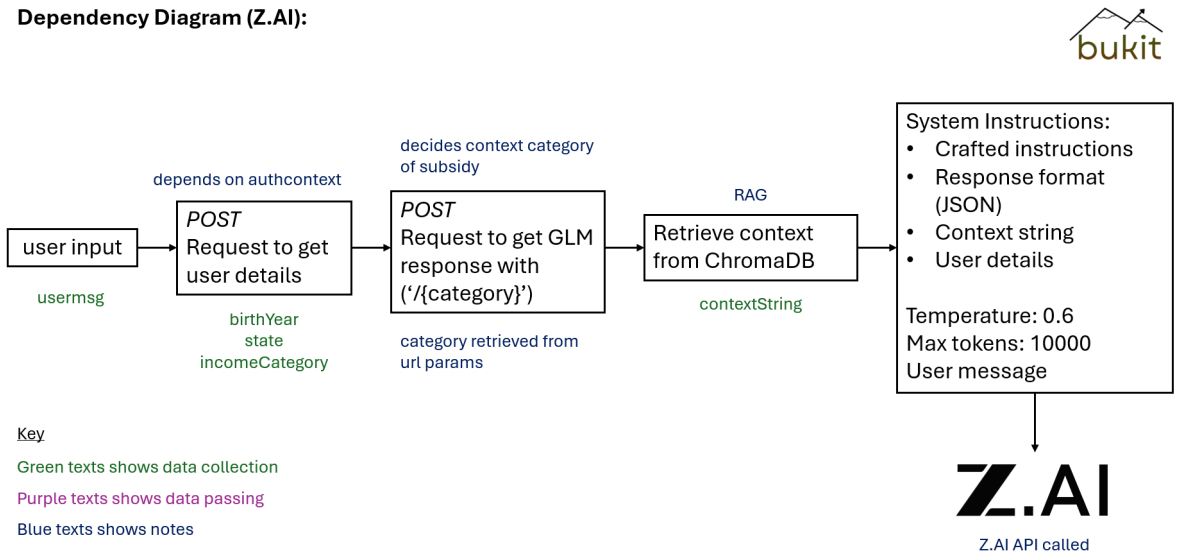
#### 2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer-not limited to a generic "AI Module". The following Dependency diagram includes:

© UMHackathon 2026

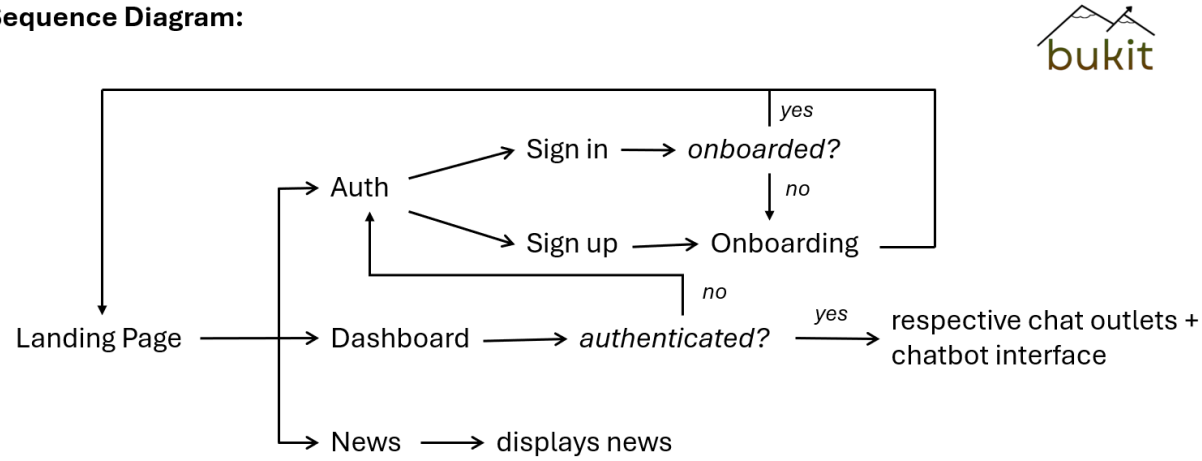
### 2.1.3 Dependency Diagram

**Dependency Diagram (Z.AI):**




### *2.2.2. Sequence Diagram*

**Sequence Diagram:**




### *2.2. Technological Stack*

2.2.1. Frontend: **React (Vite)** for web-application

2.2.2. Backend: **Node.js (Express)** for RESTful APIs

2.2.3. Database: **MongoDB** for user authentication details and user profiles, **ChromaDB** for vector embeddings for RAG.

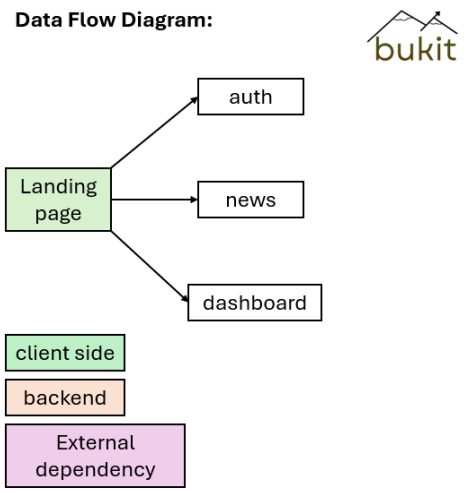
2.2.4. Cloud/Deployment: Deployed via **Vercel**

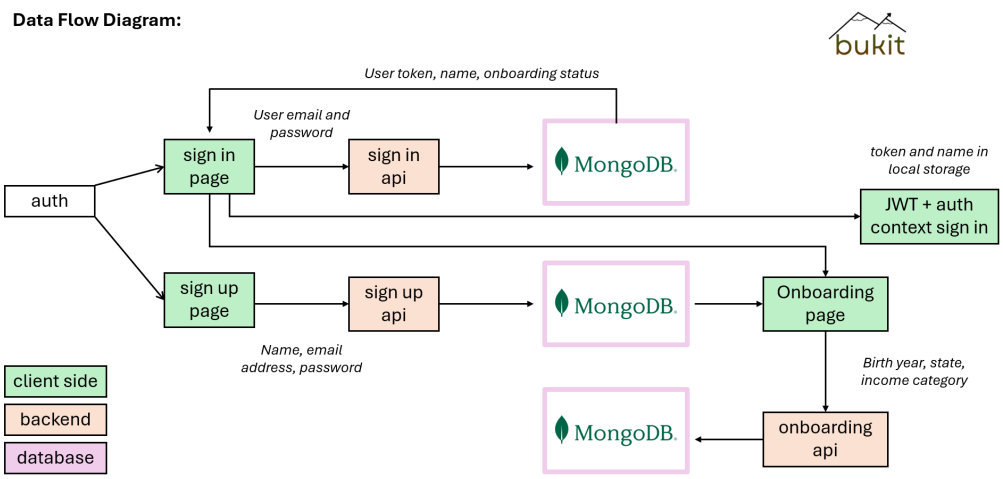
### *2.3. Key Data Flows*

© UMHackathon 2026

*2.3.1. Here, do show the how data are moving through the system and how its structure and stored*

***2.3.1.1. Data Flow Diagram (DFD)***



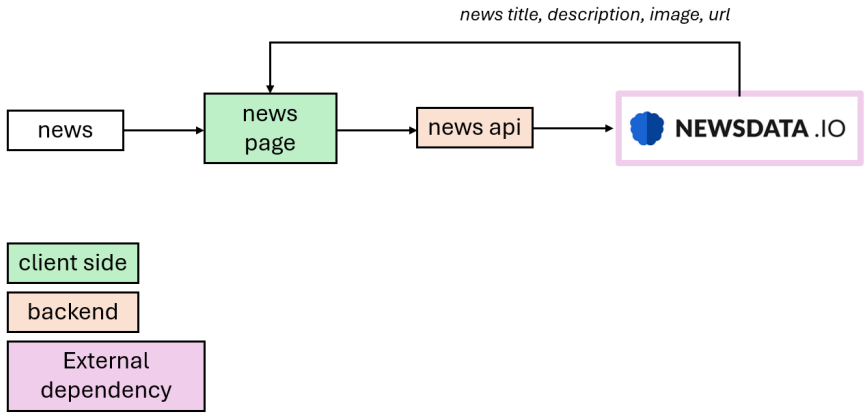




© UMHackathon 2026

Email: umhackathon@um.edu.my

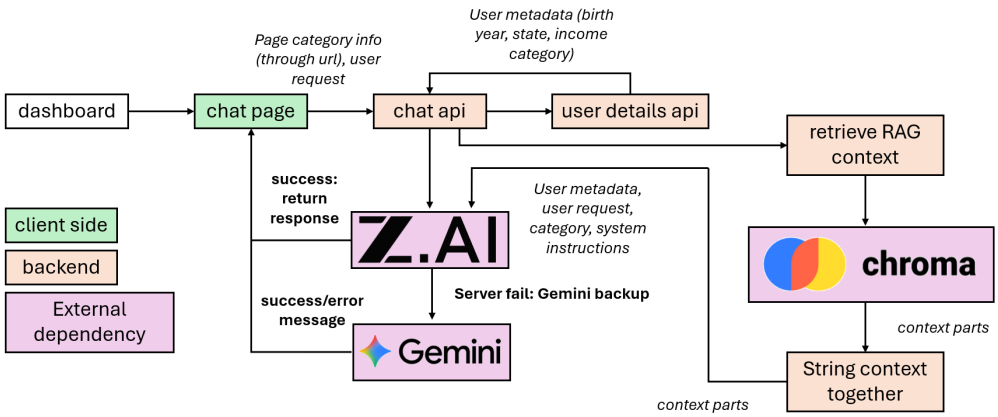
**Data Flow Diagram:**

bukit




**Data Flow Diagram:**

bukit

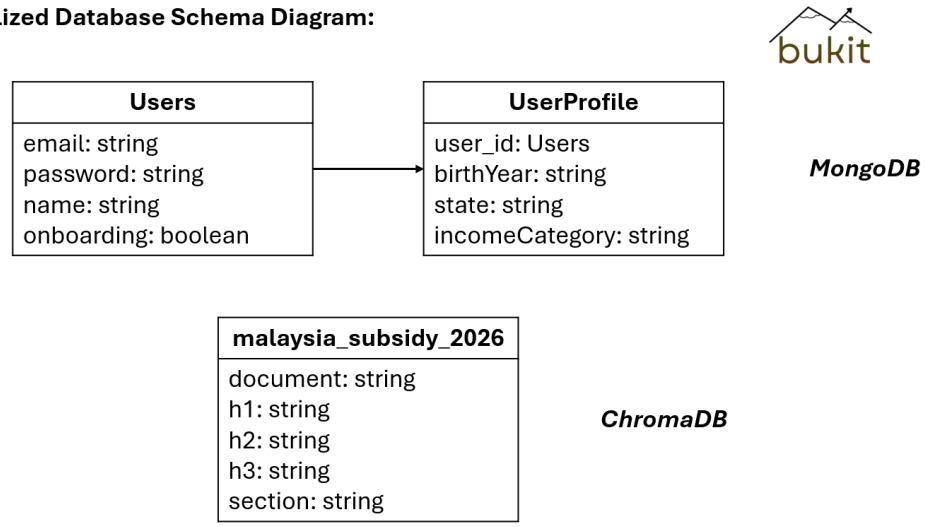



© UMHackathon 2026

Email: umhackathon@um.edu.my

### 2.3.2. Normalized Database Schema

**Normalized Database Schema Diagram:**




## 3. *Functional Requirements & Scope*

*This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.*

## 3.1. *Minimum Viable Product:*

***3.1.1.*** *Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.* ***[e.g. in Table]***

| *#* | *Features* | *Description* |
|---|---|---|
| *1* | Secure Authentication and Contextual Onboarding | A user authentication flow with encrypted JWT, also featuring a sequence where the system captures and stores the crucial data like birth year, state and income category to personalize the users' future AI interactions. |
| *2* | Smart Subsidy Matching Engine | ILMU-GLM-5.1 utilizes context injected prompting to match users with specific subsidies (e.g., MyKasih, BUDI Sabah) based on their stored onboarding profile. |
| *3* | News Module | A dedicated page that pulls in current subsidy updates for Malaysian citizens. |
| *4* | Categorized Subsidy Dashboard | A discovery page that organizes financial aids into 6 sections: Utilities, Transportation, Healthcare, Education, and Groceries. This allows the users to |

© UMHackathon 2026

| | | |
|---|---|---|
| | | browse general aid information visually and easily before moving into a personalized chat. |
| ***5*** | High-Impact Landing Page | A professional entry point that could gain trust by highlighting the platform's core values: real time subsidy tracking, enforced data privacy and a deep focus on the Malaysian economic context (e.g., MyKasih and RON 95 petrol subsidies). |

## *3.2. Non-Functional Requirements (NFRs)*

*Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]*

| ***Quality (e.g.)*** | ***Requirements (e.g.)*** | ***Implementation (e.g)*** |
|---|---|---|
| *Scalability* | *Ability to handle a high number of traffic of users on the website.* | *Automatic scaling to be implemented for MongoDB (to ensure user database size can be vertically scaled).* |
| *Reliability* | *Ensure 99% of chats should be able to return a response.* | *Maintain consistent availability of credits in Z.AI and available quota in rate limits for Gemini* |
| *Maintainability* | *Separated code base for frontend and backend, ensuring readability.* | *Independent CI pipeline to test automatic build, lint, and unit tests, along with git leaks to prevent API keys leakage.* |
| *Token Latency* | *API calls to GLM must return within 30 seconds, and chat responses be returned to the user within 90 seconds.* | *GLM must return a response within 30 seconds, or else it will be timed out and passed over to Gemini API for fallback.* |
| *Cost Efficiency* | *Average cost for users per session should be under 10k tokens.* | *Capping max tokens for Z.AI to be under 10k.* |

## *3.3. Out of Scope / Future Enhancements*

*Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026. [e.g. below]*

© UMHackathon 2026

a. *Multi LLM cross checked automated RAG pipeline for gathering latest subsidies and agentic creation of markdown files to store RAG context.*
b. *Allows followup questions from GLM/LLM to better understand the user's question.*
c. *Ability for users to attach files or images to GLM through information extraction from other LLM models.*
d. *Integration of OpenFinance API to evaluate financial documents and provide a more in depth user profile, leading to better suggestions provided based on the context.*

## 4. Monitor, Evaluation, Assumptions & Dependencies

### 4.1. Technical Evaluation:

#### 4.1.1. Grayscale Rollout & A/B Testing: *Do highlight team's strategy of deployment*

Rolling out to 10 beta testers initially, monitoring logs and fixing any bugs. Gradually increased number of testers, if no issues are detected. Collecting verbal feedback for A/B testing from beta testers.

#### 4.1.2. Emergency Rollback (ER) & Golden Release: *Do mention the strategies for Emergencies.*

Emergency taskforce to debug raised error upon beta tester's notification, rolling back to the previous bug-free deployment.

#### 4.1.3. Priority Matrix:

4.1.3.1. *Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project. [e.g. P1 Critical: If payment failure error is more than 1%, emergency rollback to Golden release.]*

P1 Critical: If chat repeatedly unable to provide response, emergency taskforce to ensure troubleshooting, raise error to Z.AI GLM, and recheck Gemini rate limits
P2 Critical: If the user profile creation (authentication) or login fails consistently, investigate MongoDB connection issues.

### 4.2. Monitoring:

4.2.1. *Show the planning for your monitoring plan of the system and how it will trigger if any damage is detected of the health of the system.*

- Vercel will send notifications upon server deployment failure when new changes are merged onto main.
- Vercel is able to track logs invoked upon error during live user activity - constantly monitoring logs to ensure the system is running as expected.

© UMHackathon 2026

- GitHub CI pipeline (detailed in Testing Analysis Document) implemented to ensure deployable code before proceeding with CD pipeline.

**4.3.** ***Assumptions:*** *State key operational & environmental conditions that you assumed to be valid while development of system and deployment.*

**a.** Users have stable internet connections on their devices during the browsing of the web page.

**b.** Users have their own devices to access an up to date search engine.

**c.** Users are capable of navigating an English based website.

*4.4.* ***External Dependencies:*** *Do list all the external tools that are applied to your system to function. [e.g. below]*

| *Tools* | *Purpose* | *Risks* |
|---|---|---|
| *Z.AI GLM API* | It should be the core LLM reasoning engine - that will handle the prompt inference, multi-step reasoning and generating structured JSON output | High- in case of GLM API rate limitation and 504 gateway timeout error may cause app's core function to not work. |
| *Gemini API* | Fallback LLM in case Z.AI GLM fails, will take over and handle the prompt inference, together with multi -step reasoning and generating structured JSON output. | Medium - fallback plan. If it comes to use, both Z.AI and Gemini failing will crash the app's core function. |
| *NewsData.io API* | *Retrieving latest Malaysian news related to subsidy from the web* | *Medium level- API free tier limitation to 200 requests per day, high traffic may cause other users to not be able to access the news anymore.* |

## *5. Project Management & Team Contributions*

*The duration of the Preliminary round is approximately 10 days considering the two-week sprint.*

*5.1.* ***Project Timeline:*** *Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]*

© UMHackathon 2026

| ***Day*** | ***Agenda*** |
|---|---|
| *Day 1 (16/4/26)* | *Project Planning - Topics, Roles* |
| *Day 2 (17/4/26)* | *Decided on layout of the pages, setup MongoDB* |
| *Day 3 (18/4/26)* | *Finish setup MongoDB, Frontend pages and components (Navigation Bar, sign in / sIgn up page)* |
| *Day 4 (19/4/26)* | *Setup the Auth Context, starting news page, dashboard page* |
| *Day 5 (20/4/26)* | *Completed news page and dashboard page, setup snackbar for sign in / sign up page, finish the landing page* |
| *Day 6 (21/4/26)* | *Meeted advisor, starting the documentation* |
| *Day 7 (22/4/26)* | *Deployed on Vercel, starting on the main prompt page* |
| *Day 8 (23/4/26)* | *Implemented the RAG pipeline, documentation, finished the main prompt page* |
| *Day 9 (24/4/26)* | *Documentation, CI Pipeline* |
| *Day 10 (25/4/26)* | *Documentation, record and edit video, Pitch Deck, submit* |
| *Day 11 (26/4/26)* | - |

5.2. ***Team Members Role:*** *Highlight the role of team members [e.g. Person A – Role Backend Lead – Fast API Microservices & Schema Design]*

| *Team Member* | *Role* |
|---|---|
| *Ryan Ng Ka Wai* | *Backend - Implementing most backend*<br>*Frontend - Helping with Auth context, Snackbar etc*<br>*Documentation - Sys Ana Doc, Pro Req Doc, Sam Test Ana Doc* |
| *Jayden Yew Jae Han* | *Frontend - Setup Navigation bar, Landing* |

© UMHackathon 2026

| | |
|---|---|
| | *page, Main prompt page, Dashboard etc*<br>*Backend - Setup pipelines*<br>*Documentation - Samp Test Ana Doc*<br>*Pitch Deck* |
| *Lee Wen Le* | *Frontend - Setup auth page, News page, Main prompt page etc*<br>*Documentation - Pro Req Doc, Sys Ana Doc*<br>*Pitch Deck* |

**5.3. Recommendations:** *If any recommendation is applicable in terms of scalability, performance and reliability [e.g. using Redis Cache for Menu data]*

- Caching news data and stored in a database on a 24 hour interval so that minimal number of requests are made to the Newsdata.io API.
- Considering utilizing Cloudflare for static assets to improve frontend loading performance and reduce latency for geographically dispersed users.
- Server side caching for frequently accessed non-user-specific data to reduce database load and API latency.

© UMHackathon 2026